CPUTrade: presenter guide (one presenter, 9 min 30 s)

One person drives the demo and does the talking. Target finish: **9:30**, leaving 30 seconds under the 10-minute limit. Lines in quotes are *suggestions*; say them in your own words, don't read them out.

Brief requirement: the video rules say *all group members should participate and be visible*. So the other two members appear on camera at the **start** (0:00-0:30) and the **end** (9:00-9:30) and each says a sentence. The main presenter does everything in between.

Before you record (10 minutes)

1. Start the app: `.\demo\start-demo.ps1` (wait about 20 seconds). Open http://localhost:5173. The app now uses the normal `cputrade` database, which holds only the demo data.
2. Use **three browser windows** so you never log in and out on camera. Put each in phone view (F12, device icon, iPhone 12 Pro 390x844):

Window	Logged in as	Used for
Chrome (normal)	`thabo_s` (buyer)	marketplace, chat, buying, reporting
Chrome Incognito	`lerato_m` (seller)	replying to chat, marking sold
Edge	`admin` (password `Admin123!`)	admin moderation

Student passwords are `Password123!`.

3. In every window, **click the X on the "Enable notifications" banner**.
4. Rehearse once with a timer. After a rehearsal, put the demo back with `mysql -u root < demo\reset-live-demo.sql`, then refresh every window.
5. Record screen plus sound, with a small webcam box so all three of you can be seen at the start and end.

If something breaks: say "let me refresh that", press F5, keep going. If the data looks wrong, run the reset file. If the whole database is damaged: stop the backend, run `mysql -u root < demo\demo-data-backup.sql`, restart the backend.

Timeline

Time	Segment
0:00-0:30	Intro: all three on camera
0:30-0:55	The problem
0:55-1:50	Marketplace tour
1:50-2:25	Listing detail and selling
2:25-2:55	Trust: CPUT-only registration
2:55-3:30	Chat between buyer and seller
3:30-5:10	The verified sale: sold, confirm receipt, rating
5:10-5:35	Community bulletin board
5:35-6:35	Safety: report and admin moderation
6:35-7:10	Under the hood: security and testing
7:10-9:10	**What is not in the system, and why (2 minutes, scripted below)**
9:10-9:30	Close (the other two on camera again)

Running late? Cut: the price filter in the tour, the "Remove" point in admin, the bulletin post. **Never cut the 2-minute "not in the system" segment**: there is no live Q&A in the video, so this is where you answer the examiner's questions before they are asked.

Segment by segment

0:00-0:30, intro *(all three on camera)*

- Each member says their name and role in one sentence.
- Main presenter: "CPUTrade is a mobile-first campus marketplace where CPUT students and approved local vendors buy, sell and swap safely."

0:30-0:55, the problem *(still on camera or title screen)*

- Three points: students trade on WhatsApp and Facebook groups; nothing is verified so scams are easy; there is no single place built for campus life (textbooks, res items, Lost and Found).
- "So we built a marketplace where identity is real and every sale is verified."

0:55-1:50, marketplace tour *(Chrome, `thabo_s`, Home)*

1. Point at the **Home screen**: the live "Just sold" ticker and the real photos.
2. Tap a **category pill** (Electronics), then the **Campus filter** (Bellville): "Five CPUT campuses, so you see what is near you."
3. Type "calculator" in **Search**, set a max price, then clear it.

4. Tick **Open to Trade**: "Sellers can offer a swap, for example this calculator for a Programming textbook."
5. Show the **R0 listing**: "Price zero shows a Freecycle badge, which supports sustainability."

1:50-2:25, listing detail and selling

1. Open the **calculator listing**: photos, condition, campus, seller rating, people watching, **Share to WhatsApp**.
2. Open **+ New Listing** (do not submit): "Listing takes under a minute: up to 6 photos, up to 4 MB each, quantity, condition and campus."

2:25-2:55, CPUT-only registration *(new tab, `/Login`, Register)*

1. Choose Register, role Student, email `test@gmail.com`: it is **rejected**. "Students must use a CPUT email."
2. Switch the email to `newstudent@mycput.ac.za` (it would be accepted).
3. "Vendors can use any email but cannot sell until an admin approves them. That is our answer to fake businesses."

2:55-3:30, chat *(Chrome `thabo_s`, then Incognito `lerato_m`)*

1. As `thabo_s`: Chats, open the Red Kanken backpack conversation, send "Can we meet at the library at 13:00?"
2. Point out read receipts, the typing indicator and safe meet-up suggestions.
3. Switch to Incognito `lerato_m`: Chats, reply "Yes, see you there."
4. "Every chat is tied to a listing, so there is a record."

3:30-5:10, the verified sale: the key moment

1. As `lerato_m`: open the **Red Kanken backpack**, click **Mark as Sold**, pick **thabo_s**. "She can only choose someone who actually messaged her about this item."
2. Switch to `thabo_s`: open the listing. He sees *"Confirm you received it to unlock leaving them a rating."* Click **Confirm I received this**.
3. The **rating form** appears: choose **5 Excellent**, add a comment, submit.
4. Show the seller's updated rating.
5. "A seller cannot rate their own sale. A buyer cannot rate before confirming receipt, and cannot rate twice. That is how we stop fake reviews."

5:10-5:35, community bulletin board *(Chrome, Board)*

- Show Lost and Found posts and the filters. "It is more than a shop: events, study groups and Lost and Found make it a community hub."

5:35-6:35, safety and admin *(Chrome `thabo_s`, then Edge `admin`)*

1. As `thabo_s`: open a listing that is not his, click **Report**, choose **Scam / fraud**, submit.
2. In Edge as **admin**, open **Admin**. Show the **Reports Queue** and review or dismiss the report.

3. In **Pending Vendor Requests**, click **Approve** on `quickfix_it`.
4. Point at **Manage Students and Vendors**: Suspend. "Suspension works immediately, even for someone already logged in, and they can appeal."
5. Point at **All Listings (Moderation)**: Remove.

6:35-7:10, under the hood *(talk over the app)*

- Tech: Vue 3 installable mobile app (PWA) with push notifications, Spring Boot API, MySQL database.
- Security: bcrypt password hashing, login tokens that expire after 24 hours, role-based permissions, login rate limiting (5 wrong passwords locks that login for 15 minutes), images checked by content, not file name.
- Quality: **25 automated tests** pass. Our concurrency test found a **real bug**: six simultaneous sales only reduced stock by one. We fixed it with a database lock and re-tested. "Testing found it before users did."

7:10-9:10, what is not in the system, and why (2 minutes)

Why this segment exists: the brief mentions payments, escrow, AI fraud detection, two-factor login and microservices. We chose not to build them, so we say so, with the reasons, before anyone asks. On screen: stay on the Home screen, or show the "brief versus built" table in Appendix A.

Say it in your own words. Suggested script (about 275 words, about 2 minutes). Each paragraph is one topic with its time budget:

1. **Opening (15 s).** "Before we finish, here is what is deliberately not in this version, and why. Being clear about limits is part of building responsibly."
2. **Payments and escrow (30 s).** "We did not build a cart or card payments. That was a decision, not an oversight. With no payments, CPUTrade never holds card details, so there is no card-data breach risk and no payment compliance burden. Instead, buyer and seller agree in chat and pay in person, and the buyer confirmation step stops fake reviews. It is not escrow, because we hold no money. A hosted PayFast checkout is our first next step, and real escrow comes with it."
3. **Fraud detection and two-factor login (25 s).** "We did not build AI fraud detection or two-factor login. With no transaction history there is nothing meaningful to train a model on, and with no money moving there is no high-value purchase for two-factor to protect. Fraud is covered by rule-based controls you have seen: CPUT-only emails, vendor approval, reports, suspensions, rate limiting, and ratings that need a confirmed receipt."
4. **Email verification (20 s).** "We check that a student uses a CPUT email domain, but we do not yet send a verification link, so someone could use another person's address. Reports and suspensions contain that risk, and email verification is next."
5. **Architecture and scale (20 s).** "It is a modular monolith, not microservices, and it runs locally rather than on cloud auto-scaling or Redis. For a team of three and a campus-sized user base that is simpler and more reliable. The API is stateless, so it can scale out, and the modules can be split into services later."
6. **Privacy and testing (10 s).** "We have no account deletion or privacy policy yet, and no independent penetration test. Those are on the roadmap before any public launch."

Safe words: say *buyer-confirmed receipt* (not escrow), *rule-based controls* (not AI), *modular monolith* (not microservices), *students must use a CPUT email* (not verified students). The full reasons and security evidence are in the appendix, if you want to rehearse deeper answers.

9:10-9:30, close *(the other two members back on camera)*

- Presenter: "Our next steps are email verification, a payment gateway, two-factor login for high-value purchases, and cloud storage."
- Member 2: one short sentence on what they contributed or learned. Member 3: the same.
- Presenter: "CPUTrade makes campus trading affordable, sustainable and safe. Thank you."

Feature cheat sheet

Area	What it does
Accounts	Student (CPUT email only), Vendor (admin approval), Admin
Listings	Up to 6 photos (4 MB each), quantity, condition, 5 campuses, Freecycle R0, swap
Discovery	Search, category, campus, price, "Open to Trade", recently-sold ticker
Chat	Per listing, read receipts, typing, last seen, edit/delete, meet-up suggestions
Trust	Seller picks the buyer; buyer confirms receipt; then one rating per sale; Verified Seller badge
Engagement	Wishlist (price-drop and sold alerts), saved searches, push notifications, seller dashboard
Safety	Report, block, admin moderation, suspensions with appeals
Community	Bulletin board with Lost and Found, WhatsApp share
Platform	Installable on phones (PWA), offline cache, works down to 320 px wide
Security	bcrypt, JWT, roles, rate limiting, safe image checks, production secrets check

Appendix: reasons, security evidence and detailed answers

Reference material. The 2-minute "not in the system" segment above is the short version of Appendix B.

CPUTrade: likely questions, honest answers and security reasons

Purpose: so you never lose marks for something you did not build, and never claim something you did not build. Every statement below is true of the current system (checked against the code on 9 Oct 2026).

The one rule

Never claim a feature that does not exist. An honest gap with a good reason scores better than a false claim that an examiner disproves in one click. Use this four-step answer every time:

1. **Acknowledge** it plainly: "We did not build X."
2. **Reason**: why (scope, time, risk, security).
3. **What we did instead**: the control that covers the same need.
4. **Next step**: how we would add it.

Be ready to explain any screen or feature you show, and be truthful about how the system was built if you are asked.

A. The brief versus what we built

Brief item	Status	What we have	Honest reason
Multi-role accounts	Built	Student, Vendor, Admin with different permissions	
Product listings	Built	Up to 6 photos, quantity, condition, campus, swap, Freecycle	
Search and filters	Built	Text, category, campus, price, "Open to Trade"	
Reviews and ratings	Built	One rating per sale, only after the buyer confirms receipt	
Notifications	Built	In-app centre plus Web Push to the device	
Community bulletin board	Built	General, Lost and Found	
Student authentication via university email	**Partly**	Only `@mycput.ac.za` / `@cput.ac.za` can register as a student	We do not yet send a verification link, so ownership of the mailbox is not proven
Vendor verification	**Partly**	Vendors cannot do anything but browse until an admin approves them	Manual check; no automatic business-registration lookup
Moderation and flagging	Built	Reports, blocking, listing removal, suspensions, appeals	
Mobile app and/or web portal	Built	Installable web app (PWA), mobile-first	
Cart and checkout	**Not built**	Buyer and seller agree in chat, then mark as sold	Deliberate: see question 1
Payment integration (PayFast, SnapScan)	**Not built**	In-person payment with buyer confirmation	Deliberate: see question 1
Escrow for expensive items	**Not built (partial control)**	Buyer-confirmed receipt gates ratings	No money is held, so it is not escrow
Two-factor authentication	**Not built**	Rate limiting, bcrypt, token expiry	See question 4
AI-driven fraud detection	**Not built**	Rule-based controls, reports, moderation	See question 3
Microservices architecture	**Not built**	Modular monolith (10 feature modules)	See question 5

Brief item	Status	What we have	Honest reason
Cloud hosting, auto-scaling, load balancing	**Not built**	Runs locally; deployment guide written	See question 6
Redis caching	**Not built**	Database queries, browser offline cache	See question 6
HTTPS and encryption	**Partly**	Passwords hashed; HTTPS is set up at deployment	Local demo runs on plain HTTP
Regular penetration testing	**Not done**	Our own security checks and 25 automated tests	See question 9

B. Likely questions and solid answers

Payments and money

1. "Why is there no cart, checkout or payment gateway?"

- *Reason (scope):* the group is three people on a fixed timeline; a real gateway also needs a merchant account and live credentials we do not have.
- *Reason (security):* by not taking payments, the system **never receives, stores or processes card data**. That removes the biggest breach risk and the compliance burden that comes with card data (PCI DSS) and with financial regulation.
- *What we did instead:* buyer and seller agree the price in a private chat, meet on campus using the safe meet-up suggestions, and pay in person. The seller marks the item sold to a named buyer, and the **buyer confirms receipt** before any rating is allowed.
- *Next step:* integrate PayFast with a hosted checkout page, so card details go straight to the gateway and never touch our server.

2. "Is the buyer confirmation an escrow?"

- No, and we do not call it one. Escrow holds money; we hold no money. It is a **trust control**: a rating cannot exist unless the named buyer confirmed they received the item, so sellers cannot write their own reviews and fake reviews are blocked.
- *Next step:* real escrow arrives with a payment gateway.

Fraud and accounts

3. "The brief asks for AI fraud detection. Where is it?"

- We did not build machine-learning detection. With no real transaction history there is no meaningful data to train on, and a poorly trained model would raise false alarms against real students.

- *What we did instead (all real and demonstrable):* CPUT-only student email rule; vendor approval; reporting by users; admin queue; blocking; suspensions that take effect on the very next request, with appeals; **login rate limiting**; rating rules that stop self-rating, double rating and rating before confirmed receipt.
- *Next step:* rule-based alerts first (for example many reports against one account, or a brand-new account posting many listings), then models once there is data.

4. "Why no two-factor authentication?"

- No sensitive money moves through the system, so the highest-risk action 2FA protects (high-value payments) does not exist yet. The brief ties 2FA to high-value purchases.
- *What protects accounts now:* bcrypt password hashing, minimum 8-character passwords, 24-hour token expiry, and rate limiting (5 wrong passwords locks that username from that address for 15 minutes, HTTP 429).
- *Honest limit:* password rules do not enforce complexity. *Next step:* 2FA (one-time code or authenticator app) once payments exist, plus email verification.

5. "How do you know a student really owns that CPUT email?"

- Honestly: we check the **domain** (and reject lookalike domains such as `mycput.ac.za.evil.com`), but we do not yet email a verification link, so a person could register with someone else's address.
- *Why it is contained:* reports, blocking, suspension and appeals; ratings only count after buyer-confirmed receipt; rate limiting.
- *Next step:* email verification with a link. This needs an email provider account, which was out of scope for us.

Architecture and scale

6. "The brief says microservices, cloud, auto-scaling and Redis. Is this scalable?"

- It is a **modular monolith**: one deployable application split into clear modules (user, product, chat, trust, rating, notification, bulletin, admin and others). For a team of three and a campus-sized user base, this is faster to build and test and has fewer failure points than a dozen services. Modules have clear boundaries, so they can be split into services later.
- *Scalability facts:* the API is stateless (JWT, no server sessions), so several copies can run behind a load balancer. We are honest about two things that must move to shared services first: the login rate limiter (in memory) and photo storage (local disk, to be object storage).
- *Caching:* no Redis. We rely on the database and a browser-side offline cache for the feed. Redis is a measured next step if load requires it.
- *Hosting:* we ran it locally; DEPLOYMENT.md documents the environment variables, HTTPS and production settings for a real host.

7. "What happens if two people buy at the same time?"

- We found a real race during testing: six simultaneous "mark sold" requests reduced stock by only one. We fixed it with a database row lock and re-tested (stock went from 50 to 44 as expected). Duplicate-submission errors now return a clean 409 message.

Data and security

8. "What personal data do you hold and how is it protected?"

- Stored: username, email, **hashed** password, chat messages, listings and photos, ratings, and (optionally) a push-notification subscription. **No card data, no ID numbers.**
- Protections: passwords never stored in plain text (bcrypt); role-based access (admin pages are admin-only; admins cannot be self-registered); suspended and unapproved accounts are blocked on every request, not just at login; a user can only read their own conversations and edit their own messages (every chat endpoint checks the user is a participant); secrets are read from environment variables, and a production-mode startup check **refuses to start** if the dev secrets are still in use.
- *Honest limits (say these before being asked):* there is no self-service account deletion or data export yet, and no formal privacy policy. For South Africa's POPIA, the next step is a privacy policy, a retention rule and a deletion process.

9. "Did you do penetration testing?"

- No professional or third-party penetration test. What we did do: automated tests for the security rules (email domain, vendor approval, password hashing, token validity, expiry and forgery, image validation, rate limiting, production secrets check, 25 tests in all), plus live attack-style checks: brute-force login (locked out on the 6th attempt), oversized and fake image uploads (rejected), and the concurrency test above.
- *Next step:* an independent penetration test before any public launch.

10. "How do you protect against common web attacks?"

- *SQL injection:* all queries go through the JPA layer with bound parameters. We use no string-built SQL.
- *Cross-site scripting (XSS):* the Vue front end escapes everything it displays, and the code never uses `v-html`.
- *CSRF:* the API uses Bearer tokens in a header, not cookies, so browsers do not attach credentials automatically; CORS allows only the configured front-end origin.
- *Malicious uploads:* we check the file's actual header bytes (JPEG or PNG only, not the file name), enforce a 4 MB limit, and store files under random names.
- *Brute force:* login rate limiting.
- *Known trade-off (be upfront):* the login token is kept in the browser's local storage, which is simple and works for a PWA but is exposed if an XSS flaw ever existed. That is why we escape all output; httpOnly cookies are a future hardening step.

11. "Dev secrets are in the code. Isn't that a risk?"

- They exist only so the project runs on a laptop. In production mode (`SPRING_PROFILES_ACTIVE=prod`) the app **refuses to start** if the JWT secret, database password or push key still has the dev value, or if the JWT secret is shorter than 32 characters. This is covered by automated tests. DEPLOYMENT.md lists what to set.

Process and quality

12. "How did you test?"

- 25 automated backend tests, all passing; a CI workflow set up to build and test on every push (written and in the repository, but we have not yet seen it run on GitHub); a manual test table of 15 scenarios covering registration, listing, chat, the verified sale, ratings, moderation and suspension; and a real defect found and fixed (question 7).
- *Honest limit:* the marketplace, chat and admin flows are tested manually, not automatically.

13. "Why is the database set up automatically instead of migrations?"

- Hibernate creates the tables for fast iteration in a student project. Documented next step: Flyway migrations before a second environment or developer, so schema changes are reviewable and repeatable.

14. "What would you do next?"

- In order: email verification; payment gateway with hosted checkout; 2FA for high-value purchases; cloud photo storage and shared rate limiting; privacy policy and account deletion; independent penetration test; more automated tests for chat, sales and admin flows.

C. Security controls at a glance

Threat	Control	Evidence
Stolen passwords from the database	bcrypt hashing	Automated test: stored value differs from the password and still verifies
Password guessing	Rate limit: 5 failures, 15-minute lock, HTTP 429	Unit tests plus live test (attempt 6 blocked)
Fake student accounts	CPUT email domain rule incl. lookalike domains; reports; suspension	Unit tests
Fake businesses	Vendor approval; vendors blocked from writes until approved	Checked on every request
Forged or expired login tokens	Signed JWT, 24-hour expiry	Unit tests (forged, expired, wrong user)
Abuse by a logged-in user	Suspension effective on the next request; appeals	Filter runs on every request
Fake reviews	Buyer-confirmed receipt; one rating per sale; no self-rating	Rules in the rating service; database unique key
Double-submit and race bugs	Row lock on sale actions; 409 on duplicates	Live concurrency test (50 to 44)
Malicious files	Header-byte check, JPEG/PNG only, 4 MB, random file names	Unit tests plus live upload test
Admin takeover	Admin role cannot be self-registered; admin endpoints admin-only	Registration validation; security config
Secrets in production	Startup check refuses dev secrets	Unit tests
Injection and XSS	Bound parameters; Vue output escaping; no `v-html`	Code review

D. Say these limits first (before an examiner finds them)

- No payments, cart or escrow (trade in person, buyer confirmation).
- Email domain is checked but no verification link is sent.
- No AI fraud detection and no two-factor authentication.
- Modular monolith, not microservices; no Redis; not deployed to the cloud.
- No account deletion, no formal privacy policy, no independent penetration test.
- Rate limiter and photo storage are single-server for now.

E. Words to avoid and words to use

Avoid	Say instead
"Fully secure" or "unhackable"	"We implemented these specific controls and tested them"
"AI-powered fraud detection"	"Rule-based fraud controls, with ML as a future step"
"Microservices"	"Modular monolith that can be split later"
"Escrow"	"Buyer-confirmed receipt"
"Encrypted payments"	"We do not process payments, so no card data is held"
"Verified students"	"Students must use a CPUT email address"
"Production ready"	"Ready for a campus pilot, with the listed next steps"